frontend development.

Answer: To implement drag-and-drop functionality in a React application, developers have a choice between using a third-party library, such as React DnD (https://github.com/react-dnd/react-dnd), or leveraging the native HTML Drag and Drop API. While libraries can provide advanced features and abstractions, the native API is often preferred in interview scenarios or for simple use cases because it is lightweight, requires no dependencies, and demonstrates a solid understanding of browser fundamentals.

The Drag and Drop API allows developers to define draggable elements and drop zones, and to handle key events such as `dragstart`, `dragover`, and `drop`. This approach is straightforward and works well for basic drag-and-drop interactions, as shown in listing 2.7.

Listing 2.7 The drag-and-drop example

```
const DragAndDrop: React.FC = () => {
  const handleDragStart = (e: React.DragEvent<HTMLDivElement>) =>
    e.dataTransfer.setData('savedId', e.currentTarget.id); #A
  };
  const handleDragOver = (e: React.DragEvent<HTMLDivElement>) =>
    e.preventDefault(); #B
  };
  const handleDrop = (e: React.DragEvent<HTMLDivElement>) => {
    const data = e.dataTransfer.getData('savedId'); #C
    const draggableElement = document.getElementById(data);
    const dropzone = e.target as HTMLElement;
    dropzone.appendChild(draggableElement!); #D
  };
  return (
    <div>
      <h1>Drag and Drop API Demo</h1>
      <div id="draggable" draggable="true" onDragStart={handleDra
        I am draggable.
      </div>
      <div #F
        id="dropzone"
        onDragOver={handleDragOver}
        onDrop={handleDrop}
      >
        Drop Zone
      </div>
```